

Technical Specification and Algorithmic Analysis of a High-Precision Astrology Chart Application

June 8, 2026

Abstract

This document provides a comprehensive technical specification and algorithmic analysis of a web-based Astrology Chart application. The system leverages the Swiss Ephemeris ported to WebAssembly (WASM) to achieve arc-second level astronomical accuracy, underpinned by NASA's Jet Propulsion Laboratory (JPL) ephemerides. The abstract details the computational pipeline: from Terrestrial Time scaling and Julian Day conversions, through planetary and house cusp calculations using Chebyshev polynomial interpolation, to the derivation of planetary aspects and midpoints. Furthermore, it outlines the mathematical models for zodiacal projections, sidereal corrections (Ayanamsa), and the algorithmic logic rendering the two-dimensional astrological chart via the HTML5 Canvas API. The architecture represents a convergence of computational astronomy and traditional astrological methodologies within a modern client-side execution environment.

1 Introduction

The generation of a precise astrological natal chart requires the resolution of celestial mechanics problems with high mathematical rigor. Traditional approximations are insufficient for applications demanding arc-second precision. This application integrates the Swiss Ephemeris—a de facto standard in computational astrology—compiled to WebAssembly, allowing browser-side execution of JPL DE441 ephemeris data. This document dissects the operational logic, mathematical foundations, and algorithmic flows of the system.

2 Swiss Ephemeris and JPL Data Integration

2.1 JPL Ephemerides and Arc-Second Accuracy

The foundation of the astronomical calculations rests on the Jet Propulsion Laboratory's Development Ephemeris (DE), specifically the DE441 model. The JPL ephemerides are not mere analytical formulae; they are numerical integrations of the equations of motion for the Solar System, fitted to observational data (radar, laser ranging, spacecraft telemetry).

The Swiss Ephemeris compresses the massive JPL data files using Chebyshev polynomials. The specific ephemeris files utilized by the application are:

- `seas_18.se1`: Asteroids ephemeris.
- `semo_18.se1`: Moon ephemeris (requires higher polynomial orders due to the Moon's erratic orbital perturbations).
- `sepl_18.se1`: Planetary ephemeris (Sun through Pluto).

The use of these files guarantees an accuracy of approximately 0.001 arcseconds for inner planets and sub-arcsecond accuracy for outer planets over a span of 12,000 years.

2.2 Time Scales: UTC, UT, and Julian Day

Astronomical calculations require a continuous, uniform time scale. The application performs the following time transformations:

1. **Local Time to UTC:** Utilizes the ECMAScript `Intl` API to resolve the IANA timezone identifier into an offset.
2. **UTC to Universal Time (UT1) / Ephemeris Time (ET):** The Swiss Ephemeris internally handles ΔT (the difference between Terrestrial Time and Universal Time) based on historical observations and future projections, ensuring precise planetary placements.
3. **Julian Day (JD) Calculation:** The function `swe_julday` converts the Gregorian calendar date into a JD number, the standard continuous count of days since noon Universal Time on January 1, 4713 BCE.

3 Core Astronomical Algorithms

3.1 Planetary Position Calculation

The primary function `swe_calc_ut(jd, planet_id, flags)` computes planetary positions. Given the JD and the planet ID (0 for Sun, 1 for Moon, etc.), the algorithm:

1. Locates the correct Chebyshev coefficient block in the `.se1` files based on the JD.
2. Evaluates the Chebyshev polynomials to interpolate the barycentric or heliocentric coordinates.
3. Applies relativistic corrections and nutation/obliquity adjustments.
4. Returns a 6-element array containing: Longitude (λ), Latitude (β), Distance (Δ), Speed in Longitude, Speed in Latitude, and Speed in Distance.

The `SEFLG_SPEED` (256) flag is explicitly set to calculate planetary velocities, which is critical for determining retrograde motion (when Speed < 0).

3.2 House Cusp Calculation

The calculation of astrological houses depends on the obliquity of the ecliptic (ϵ), the geographical latitude (ϕ), and the Local Sidereal Time (LST). The application uses `swe_houses(jd, lat, lon, hs)`.

For the default Placidus system, house cusps are defined by the diurnal and nocturnal semi-arcs of the ecliptic. The Ascendant (ASC) is the intersection of the ecliptic with the eastern horizon, calculated as:

$$\lambda_{ASC} = \arctan \left(\frac{-\cos(\text{RAMC})}{\sin(\epsilon) \tan(\phi) + \cos(\epsilon) \sin(\text{RAMC})} \right) \quad (1)$$

where RAMC is the Right Ascension of the Midheaven. The Midheaven (MC) is simply:

$$\lambda_{MC} = \arctan \left(\frac{\tan(\text{RAMC})}{\cos(\epsilon)} \right) \quad (2)$$

The application supports over 20 house systems (e.g., Koch, Porphyry, Regiomontanus), specified by single-character codes passed to the ephemeris.

3.3 Sidereal Corrections and Ayanamsa

If a sidereal zodiac is selected (Ayanamsa $\neq -1$), the Tropical longitudes (λ_T) are converted to Sidereal longitudes (λ_S) by subtracting the Ayanamsa (A):

$$\lambda_S = \lambda_T - A \quad (3)$$

The application calls `swe_set_sid_mode` to establish the reference point (e.g., Spica at 0° Libra for Lahiri) and calculates the precise A value for the given JD. This correction is applied post-calculation to all planetary longitudes, house cusps, and the Ascendant/Midheaven.

4 Astrological Logic: Aspects and Midpoints

4.1 Aspect Calculation Algorithm

An aspect is a specific angular relationship between two planetary longitudes. The algorithm computes the absolute angular difference ($\Delta\lambda$):

$$\Delta\lambda = |\lambda_1 - \lambda_2| \pmod{360} \quad \text{if } \Delta\lambda > 180 \implies \Delta\lambda = 360 - \Delta\lambda \quad (4)$$

The algorithm then compares $\Delta\lambda$ against a predefined set of aspect angles (α). An aspect is valid if the deviation (orb, O) is within tolerance:

$$\text{IsAspect} = |\Delta\lambda - \alpha| \leq O \quad (5)$$

The application implements a hierarchical aspect filter distinguishing between Major aspects (e.g., Conjunction $\alpha = 0, O = 8$) and Minor aspects (e.g., Novile $\alpha = 40, O = 2$). The exact orb is stored for UI rendering.

4.2 Midpoint Calculation

The midpoint (λ_m) between two planets is the average of their ecliptic longitudes, adjusted for the 0/360 boundary:

$$\lambda_m = \frac{\lambda_1 + \lambda_2}{2} \pmod{360} \quad (6)$$

However, if the longitudinal distance between the planets crosses the vernal point ($|\lambda_1 - \lambda_2| > 180$), the shorter arc must be calculated, resulting in:

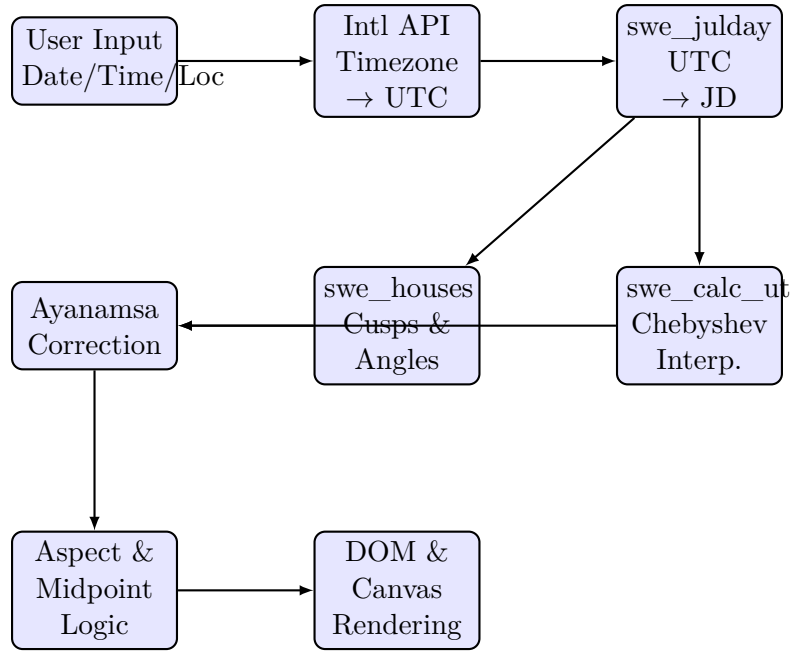
$$\lambda_m = \left(\frac{\lambda_1 + \lambda_2}{2} + 180 \right) \pmod{360} \quad (7)$$

5 System Architecture and Block Flows

The application follows an asynchronous, event-driven architecture.

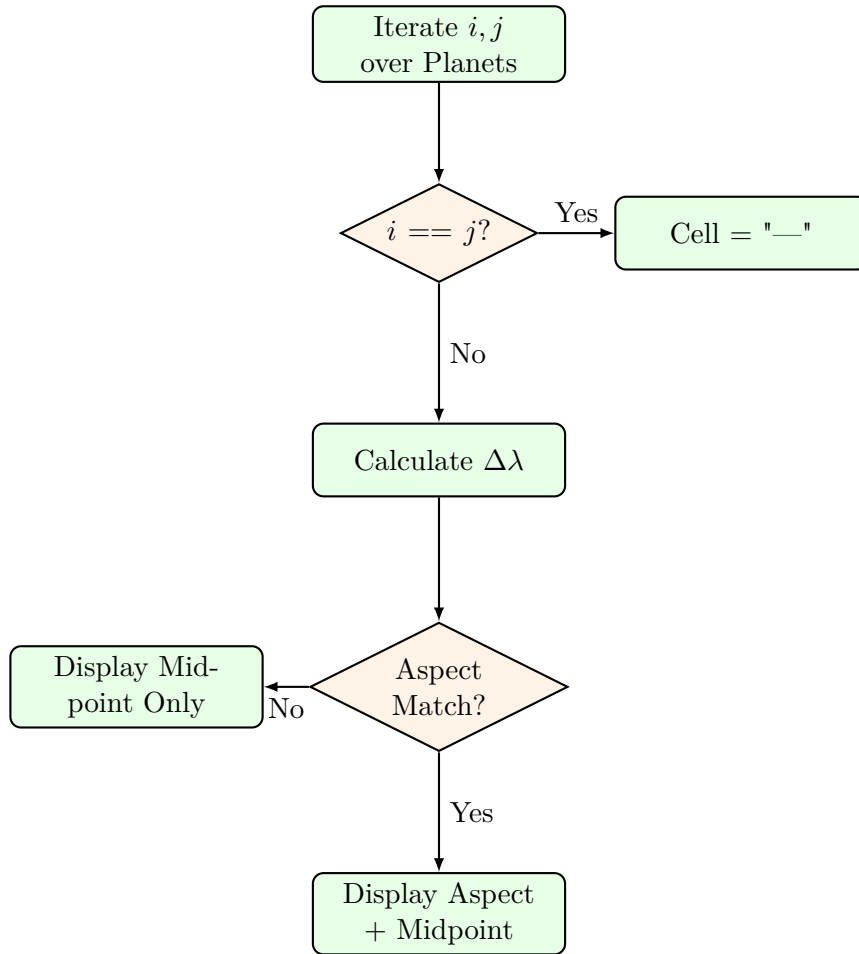
5.1 Initialization and Calculation Flow

The following diagram illustrates the primary data flow from user input to ephemeris resolution.



5.2 Aspect Matrix Generation Flow

The UI requires a symmetrical matrix mapping every planet against every other planet.



6 Chart Rendering Engine

The visual chart is rendered using the HTML5 Canvas 2D API. It requires mapping geocentric ecliptic longitudes onto a polar coordinate system centered on the Ascendant.

6.1 Coordinate Transformation

The standard mathematical polar coordinate system places 0 at the 3 o'clock position. Astrological charts place the Ascendant at the 9 o'clock position. The mapping function used is:

```
1 const toCanvasAngle = (lon) => {  
2   // ascOffset is the longitude of the Ascendant  
3   return (Math.PI / 180) * (ascOffset - lon) + Math.PI;  
4 };
```

This effectively rotates the coordinate space such that the Ascendant aligns with the leftward horizontal axis, and longitude increases counter-clockwise.

6.2 Rendering Layers

The rendering pipeline paints the chart in strict order to manage visual occlusion:

1. **Inner Background:** Central disk.
2. **Aspect Lines:** Drawn directly inside the inner disk. Minor aspects are rendered with dashed lines (`setLineDash`).
3. **Zodiac Ring:** Divided into 12 wedges of 30 each. Colored with distinct spectral mappings and overlaid with Unicode zodiac glyphs.
4. **Degree Ticks:** 360 individual ticks on the outer zodiac perimeter, with major ticks at 10 intervals labeled with their degree-within-sign value.
5. **House Ring:** Divided according to the calculated cusps. Angular houses (1, 4, 7, 10) have lines extending through the zodiac ring for emphasis.
6. **Planets:** Planets are plotted at their longitudinal positions. A collision avoidance algorithm shifts planets radially inward if their angular proximity (< 0.18 radians) would cause glyph overlap.

7 Conclusion

This application represents a sophisticated implementation of computational astronomy within a web browser. By offloading the heavy numerical integration of JPL DE441 data to a WebAssembly-compiled Swiss Ephemeris, it achieves observatory-grade arc-second precision. The algorithmic logic correctly handles the nuances of ecliptic coordinate transformations, sidereal offsets, and complex astrological matrices, culminating in a robust and mathematically precise astrological charting tool.

References

- [1] Folkner, W. M., Williams, J. G., Boggs, D. H., Park, R. S., & Kuchynka, P. (2014). *The Planetary and Lunar Ephemerides DE430 and DE441*. Interplanetary Network Progress Report, 42-196.

- [2] Astrodienst AG. (2023). *Swiss Ephemeris Programmer's Manual*.
- [3] Meeus, J. (1998). *Astronomical Algorithms*. Willmann-Bell, Inc., Richmond, VA.
- [4] Rivlin, T. J. (1990). *Chebyshev Polynomials: From Approximation Theory to Algebra and Number Theory*. John Wiley & Sons.